

Om deploy

Lær om effektiv bruk av Argo CD for å deploye applikasjoner ved å administrere manifeste og benytte beste praksiser.

Introduksjon

Marvin leveres med Argo CD, et deklarativt GitOps-verktøy som forenkler prosessen med å deploye. Med Argo CD kan du definere applikasjonens ønskede tilstand i et git-repo, og verktøyet sørger for at tilstanden i kubernetes er alltid oppdatert med tilstanden beskrevet i git.

I denne artikkelen skal vi se nærmere på konsepter som:

- Forskjellen mellom Kubernetes-manifeste og Argo CD-manifeste
- Hvordan manifeste bør struktureres
- GitOps-prinsipper og beste praksiser
- Argo CD `Application` og `ApplicationSet`

Dersom du allerede kan disse konseptene kan du hoppe rett til å [Konfigurere Application](#).

Forutsetninger

Denne artikkelen diskuterer konsepter som bygger videre på prinsipper som DevOps, GitOps, og CI/CD. Gjør deg gjerne kjent med moderne praksiser som [12-factor](#) også.

Typer Manifeste

Det finnes 2 ulike manifeste vi trenger å forholde oss til:

Type	Beskrivelse	Endringsfrekvens
Kubernetes-manifeste	Helm, Kustomize eller vanlige manifeste i Git	Svært ofte

Type	Beskrivelse	Endringsfrekvens
Argo CD-manifester	Argo CD <code>Application</code> og <code>ApplicationSet</code>	Nesten aldri

Den første kategorien er standard Kubernetes-ressurser (deployment, service, ingress, config, secrets, osv). Disse ressursene har ingenting med Argo CD å gjøre, men beskriver hvordan en applikasjon kjører i Kubernetes. Disse manifestene endres ofte, som for eksempel ved oppdatering av container-image.

Den andre kategorien er Argo CD-manifester. Dette er konfigurasjon som refererer til "sannhetskilden" for en applikasjon (kubernetes-manifester), og forklarer hvor og hvordan disse skal deployes. Argo CD-manifester kan ses på som en enkel kobling mellom Kubernetes-manifester og et namespace.

For å gjøre livet enklere er det lurt å ha et veldig tydelig skille mellom Kubernetes-manifester og Argo CD-ressurser. Argo CD har (dessverre) flere funksjoner som lar deg blande begge typene. For eksempel støtter Argo CD overskriving av helm-verdier:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: gitops-developers
spec:
  source:
    repoURL: https://github.com/example-org/example-repo.git
    targetRevision: HEAD
    path: charts/my-app/

    # DONT DO THIS
    helm:
      parameters:
        - name: "my-setting-1"
          value: my-value1
      values: |
        ingress:
          enabled: true
          path: /
          hosts:
            - mydomain.example.com
```

Dette manifestet fikses ved å følge standarden som Helm forventer. Med andre ord, legg heller verdier i `values`-filer i chartet og referer til dem direkte:

```
spec:
  source:
```

```
repoURL: https://github.com/example-org/example-repo.git
targetRevision: HEAD
path: charts/my-app/
helm:
  valueFiles:
    - values.yaml
    - environment/<miljø>/values.yaml
```

Argo CD har også muligheten til å automatisk detektere hvilket verktøy som skal deployes med. Dersom kubernetes-manifestene inneholder `Chart.yaml` brukes Helm. Hvis den finner `kustomization.yaml` brukes Kustomize. Andre kubernetes-manifester vil bli deployet med kubectl. Konfigurasjonen til "sannhetskilden" blir dermed kort og tydelig:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: gitops-developers
spec:
  source:
    repoURL: https://github.com/example-org/example-repo.git
    targetRevision: HEAD
    path: /path/to/manifests
```

Vi ønsker å oppnå at kubernetes-manifestene i seg selv er en klar indikasjon på hva som er ønsket tilstand. Dette gjør manifestene enklere å forstå og kobler ikke manifestene dine til spesifikke Argo CD-funksjoner.

For utvidet informasjon rundt hvordan manifester burde organiseres anbefales denne artikkelen: <https://codefresh.io/blog/how-to-structure-your-argo-cd-repositories-using-application-sets/>.

Argo CD Application

En Argo CD `Application` er en custom Kubernetes-ressurs som definerer hvordan en applikasjon skal deployes og administreres ved hjelp av Argo CD. Den håndterer all informasjon som kreves for å deploye:

- Hva skal deployes (`source`)
- Hvor skal det deployes (`destination`)
- Hvordan skal det oppdateres (`syncPolicy`)

La oss se på et eksempel. Her ønsker vi å deploye billing-komponenten vår til prod:

```
billing-prod.yaml
```

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: billing
  namespace: argocd
spec:
  project: default
  source:
    repoURL: 'https://github.com/my-org/billing.git'
    targetRevision: HEAD
    path: 'manifests/billing/envs/prod'
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: 'my-billing-namespace'
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

Vi velger å plassere Argo CD Applications under mappen `applications/`. Disse peker på Kubernetes-manifestene våre under `manifests/`.

Kustomize

```

├─ applications
│  ├─ billing-prod.yaml
│  ├─ billing-test.yaml
│  └─ orders-test.yaml
├─ manifests
│  ├─ billing
│  │  ├─ base
│  │  └─ envs
│  │     ├─ prod
│  │     └─ test
│  └─ orders
│     ├─ base
│     └─ envs
│        └─ test
└─ README.md

```

Helm

```

├─ applications
│  ├─ billing-prod.yaml
│  ├─ billing-test.yaml
│  └─ orders-test.yaml
├─ charts
│  ├─ billing
│  │  ├─ templates
│  │  └─ environment
│  │     ├─ prod
│  │     └─ test
│  └─ orders
│     └─ templates

```



Navnene spiller egentlig ingen rolle. Det viktigste er et tydelig skille mellom kubernetes-manifestene og Argo CD-manifestene.

I teorien trenger vi ikke å ha begge typene manifeste i samme repo engang. Dette gjør vi hovedsakelig fordi tilgangstyringen blir enklere og fordi kilden til sannhet holdes på ett sted.

I eksempelet ovenfor har vi bare miljøene prod og test. Det er opprettet Argo CD-manifeste som samsvarer med disse miljøene under `applications/`. Vi ser enkelt at `billing` er deployet til prod og test, mens `orders` er bare deployet til test.

Med denne strukturen blir vanlige scenarioer simpelt:

- Se all konfigurasjon i `test`-miljøet

```
cd manifests/billing
kustomize build envs/test
```

- Se forskjellene på ingress mellom `prod` og `test`

```
cd manifests/billing
diff envs/prod/ingress.yaml envs/test/ingress.yaml
```

- Se *alle* forskjellene mellom `prod` og `test`

```
cd manifests/billing
kustomize build envs/prod/> /tmp/prod.yaml
kustomize build envs/test/ > /tmp/test.yaml
diff /tmp/prod.yaml /tmp/test.yaml
```

Legg merke til at Argo CD ikke trenger å være involvert i noen av scenarioene. Det er nå enkelt å integrere andre CI/CD-verktøy i git-repoet ditt. Det gjør det også enkelt å automatisere opprettelsen av Argo CD-manifeste.

Argo CD ApplicationSets

Ideelt sett burde du ikke trenge å opprette Argo CD manifeste i det hele tatt. Et Argo CD `ApplicationSet` kan ta seg av opprettelsen av alle Argo CD `Applications` for deg. Dette sparer oss for mye boilerplate og utfylling av filer.

Alt vi trenger er et bindeledd mellom kubernetes-manifester og hvor det skal deployes. Dersom alle kubernetes-manifester følger en standardisert struktur kan koblingen mellom hvilket manifest som skal deployes til hvilket miljø gjøres automatisk basert på templates. Med andre ord kan behovet for mappen `applications/` fjernes.

Les eventuelt mer i den offisielle dokumentasjonen om Argo CD [ApplicationSet](#).

WIP

Det jobbes med å opprette et standardisert `ApplicationSet`. Templatene vi benytter for `ApplicationSet` utvikles av Sopra Steria ettersom disse bare kan opprettes med administrator-rettigheter. Vi har likevel mulighet til å påvirke hvordan `ApplicationSet` template fungerer gjennom dialog med Sopra Steria.

I mellomtiden anbefaler vi å bruke vanlige Argo CD Application's.

PS: Ikke fall for fristelsen om å "generere" Application-ressurser med helm. Dette vil skape mer trøbbel enn moro. Vent heller på `ApplicationSets`.

Kontinuerlig Deployment

For å kontinuerlig deploye nye versjoner av applikasjonen må det settes opp noe som kan automatisk oppdatere image-tag i kubernetes-manifestene når et nytt image er bygget.

Se eventuelt [Kontinuerlig deployment](#) for forslag på hvordan `TargetRevision` kan brukes for å peke på forskjellige miljøer.

WIP

Det jobbes med å finne løsning som automatiserer oppdatering av image-tag.

Videre lesing

- Les om [Tilgangstyring](#) for å gi Argo CD tilgang til manifestene.
- Lær hvordan du kan [Konfigurere Application](#).